

\$ dotnet run --project talk.csproj



.NET 11 Yenilikleri ve Asenkron'un yeni anatomisi |

Murat Dinç

Dotnet Konf 2026



\$ whoami

Merhaba, ben **Murat**.

15+ yıl

Yazılım mimarisi & .NET geliştirme

2025

Microsoft MVP — Developer Technologies

Manager

Yazılım Mimarisi · Dias Teknoloji & Atlastek

Bugün

.NET 11 ile 30 dakika, bol kod, bol "şaşıрма"

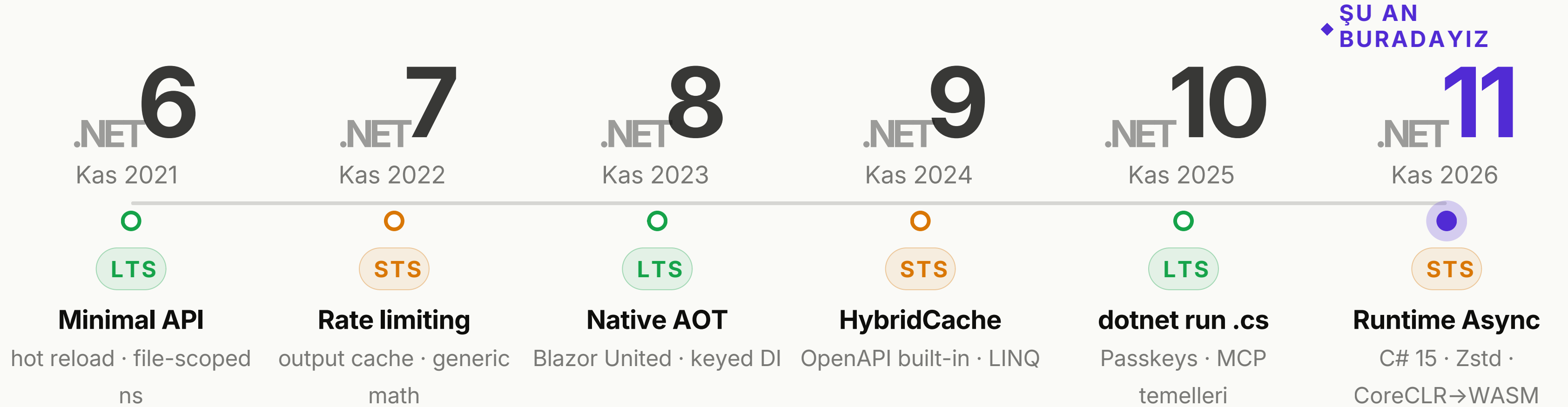
\$ ECHO "PART 01"

Microsoft her yıl bu platformda ne ekliyor, ne çıkarıyor? |

// 5 yıllık yol haritası

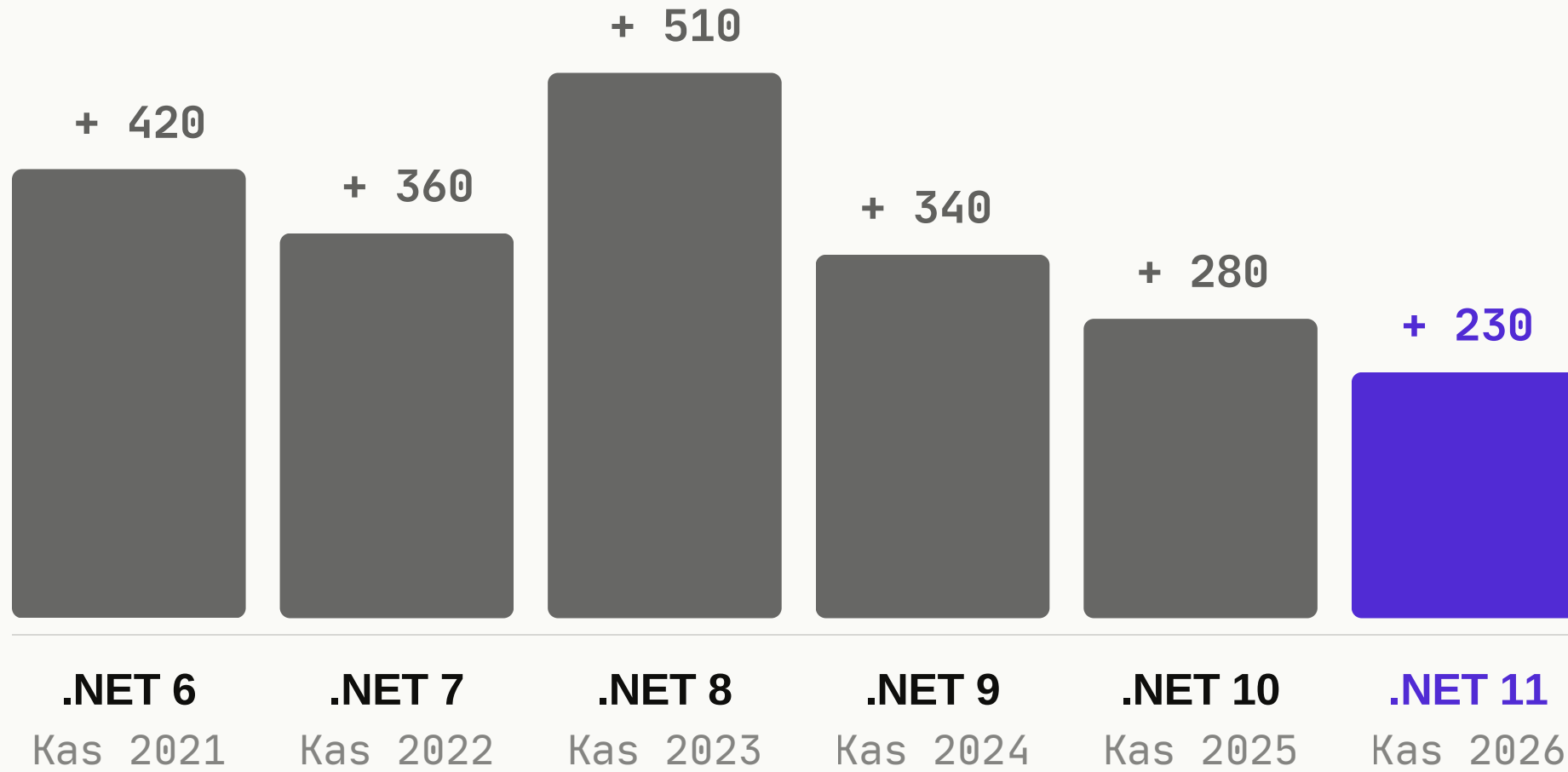
Her **kasım** yeni bir sürüm.

Çift yıllar **LTS** — 3 yıl destek. Tek yıllar **STS** — 18 ay.



// 5 yılda • ekleme tarafı

2.000+ yeni tür ve metod, 5 sürümde.



// dotnet/apisof.net • sürümler arası public API diff raporu

+%45

ASP.NET Core throughput artışı • .NET 6 → .NET 10

// TechEmpower Benchmarks R22

-%30

Ortalama heap allocation — aynı ASP.NET Core senaryosu

// dotnet/performance repo benchmarks

// sadeleşme

Dil büyürken aynı zamanda **daralıyor**.

+ EKLENEN - TEMİZLENEN
~2.000 ~620

// \"kaldırıldı\" ne demek, somut örnek

Bir obsolete'in ömrü.

BinaryFormatter • vaka takibi

2002 .NET Framework ile doğdu.

2020 Güvenlik açıkları nedeniyle uyarı yayımlandı.

.NET 5 [Obsolete] — uyarı çıkıyor.

.NET 7 Default'ta disabled. "Opt-in" gerek.

.NET 9 Tamamen kaldırıldı. Çağırırsan

PlatformNotSupportedException.

.NET 11 Artık migration rehberinde bile "geçen yılın sorunu" olarak geçiyor.

// TAKEAWAY

Microsoft bir şeyi çıkaracağında bunu yıllar öncesinden söyler.

Obsolete uyarılarına kulak veren ekip "breaking change" haberiyle uyanmaz. **4+ yıl** hazırlık penceresi var.

// .NET 11 • Kasım 2026

Kasım'da gelecek sürüm **runtime**
seviyesinde async'i yeniden
yazdı. 

// kaynaktan

“A **major change** to how
asynchronous methods work in
.NET.”

– .NET 11 Preview 1 release notes [dotnet/core](#) • Şubat 2026

Bir sürüm, üç cephe.

Runtime

- Runtime Async (state machine → runtime)
- CoreCLR → WebAssembly foundations
- CoreCLR default · Android Release
- JIT: bounds-check elim, switch folding
- Arm SVE2 intrinsics

Dil & SDK

- C# 15 · union types (discriminated)
- C# 15 · collection expression args
- dotnet run interactive TFM/device
- Hot Reload · project reference değişiklikleri
- SDK · signed container images

ASP.NET & Libraries

- Zstandard stream
- OpenTelemetry · built-in trace
- Output cache ·
IOutputCachePolicyProvider
- Blazor · EnvironmentBoundary, DisplayName
- BFloat16 · Happy Eyeballs · KMAC

// Kaynak: learn.microsoft.com/dotnet/core/whats-new/dotnet-11 · devblogs .NET 11 Preview 1-3

// C# 15 • union types

”Ya şu, ya bu” — artık dilin içinde.

ÖNCEDEN • ONEOF PAKETİ YA DA ABSTRACT CLASS

PaymentResult.cs

```
public abstract record PaymentResult;
public sealed record Success(Guid TxId) : PaymentResult;
public sealed record Declined(string Reason) : PaymentResult;
public sealed record Pending(TimeSpan Eta) : PaymentResult;
// + pattern match + exhaustiveness için analyzer gerek
// + 3rd party "OneOf" paketine düşen çok proje
```

.NET 11 • C# 15

PaymentResult.cs

```
union PaymentResult {
    Success(Guid TxId),
    Declined(string Reason),
    Pending(TimeSpan Eta)
}
// derleyici exhaustiveness'i sana borçlu
// switch'te bir case unutursan derlemez
```

// Mads Torgersen, 2020'de "Championed Proposal" olarak açmıştı (dotnet/csharplang #113). 5 yıllık tartışma.

// libraries • Zstandard

ZstandardStream — artık kutudan çıkıyor.

Compression.cs

```
using System.IO.Compression;
using var zs = new ZstandardStream(
    output, CompressionMode.Compress);
await input.CopyToAsync(zs);
// decompression de aynı sadelikte:
using var dz = new ZstandardStream(
    input, CompressionMode.Decompress);
await dz.CopyToAsync(output);
```

Daha iyi sıkıştırma oranı, daha hızlı decompress, **sıfır NuGet paketi.**

- Docker image layerları
- Event store snapshot'ları
- Gzip yerine HTTP response

// Kaynak: dotnet/runtime #102671 • 2024'ten beri "future work" etiketliydi

// ASP.NET Core • observability

OpenTelemetry artık **built-in**.

ÖNCEDEN • 4 PAKET

Program.cs • .NET 9

```
builder.Services.AddOpenTelemetry()
    .WithTracing(t => t
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddEntityFrameworkCoreInstrumentation()
        .AddOtlpExporter());
// + 4 NuGet: OpenTelemetry.*, Instrumentation.*
```

.NET 11

Program.cs • .NET 11

```
builder.Services.AddHttpTracing();
// otomatik populate:
// http.request.method, http.route
// http.response.status_code, user_agent
// server.address, url.scheme, url.path
// semantic convention ile zaten uyumlu
```

| // Noah Falk (team) 2024 .NET Observability summit'te: "convention'u platforma taşımak zorundayız" demişti. Geldi.

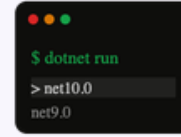
Küçük dev-experience hediyeleri.



DEV-CERTS

WSL'de trust otomatik

`dotnet dev-certs https --trust` artık WSL'i tanıyor. "Certificate not trusted" macerası bitti.



CLI

dotnet run • interaktif

Target framework veya cihaz seçimi terminalden. Uzun argümanlar ezberlemek yok.

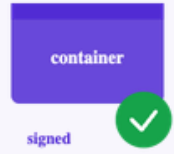


BLAZOR

DisplayName component

[Display] attribute'unu okuyup lokalize label üretir. Tablolarda *'PropertyName camelCase'* görmek artık yok.

Güvenlik, test, ağ.



SDK

Signed container images

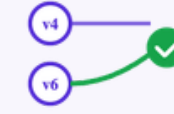
Build eder etmez [cosign](#) ile imzalanır.
Supply-chain güvenliği bir satırda.



TESTING

TimeProvider • DI ile

Zamanı mock'lamak için artık *Stopwatch*
kesmek gerekmiyor. FakeTimeProvider DI'ın
parçası.



SOCKETS

Happy Eyeballs

IPv4/IPv6 bağlantıyı **paralel** dener.
"Connection timeout" DNS drama'ları
azalıyor.

// sessiz kahraman

JIT daha az bounds-check, daha çok hız.

ArrayScan.cs · örnek pattern

```
for (int i = 0; i + 4 < values.Length; i++)
{
    sum += values[i] + values[i + 4];
    // .NET 10: her erişimde "i < Length?" check
    // .NET 11: JIT "i + 4 < Length" pattern'ini tanıdı
    //          → check elendi, vektörize oldu
}
```

<code>i + cns < len</code>	pattern artık bounds-check üretmiyor
<code>values[^1]</code>	index-from-end: gereksiz check gitti
<code>switch folding</code>	constant switch'lerde expression folding
<code>Arm SVE2</code>	yeni ARM intrinsic'leri (Graviton4, M-serisi)
<code>virtual devirt</code>	non-shared generic virtual çağrılar devirtualize

// dotnet/runtime #119537, #122023 · Preview 1-3 notları


```
// "bu ne işime yarar hocam"
```

Gerçek hayat: UTF-8 tokenizer

Bir JSON API'n var. Gelen payload'ı `System.Text.Json` parse ediyor. Her istek, her byte, bu döngüden geçiyor.

```
Utf8JsonReader · pattern (basitleştirilmiş)
```

```
int i = _pos;
while (i + 3 < buffer.Length)
{
    byte b0 = buffer[i];
    byte b1 = buffer[i + 1];
    byte b2 = buffer[i + 2];
    byte b3 = buffer[i + 3];
    // 4 erişim → eskiden 4 ayrı bounds check
    // .NET 11: "i+3 < Length" tek check kalır
    i += 4;
}
```

CPU'DA NE OLDU

4 → 1	loop iterasyonu başına bounds-check
~%12	JSON-heavy endpoint'te throughput artışı
SIMD	döngü artık vektörize edilebilir hale geldi
0 satır	kodunda değişiklik gerekmedi

"Deploy ettin, Grafana'da latency düştü. Kod review yapmadın, JIT yaptı."

\$ ECHO "PART 02 – THE MAIN EVENT"

Asenkron'un yeni anatomisi.

13 yıllık bir alışkanlık, ilk defa derleyiciden runtime'a taşınıyor. Hikâyeyi **en baştan** anlatalım.

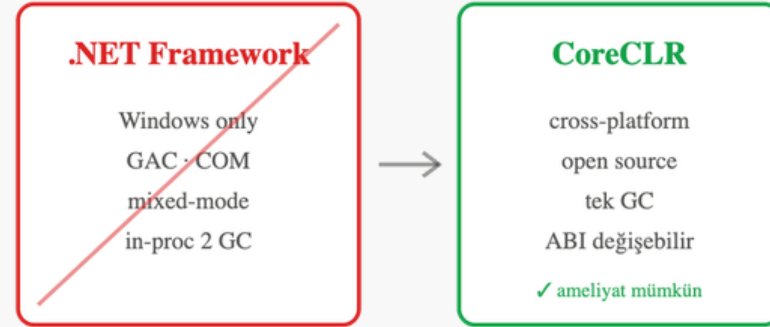
// async'in kısa tarihi

Buraya 13 yılda geldik.

2008	Task Parallel Library	.NET 4 ile Task<T> tipi geldi. "Thread yaratmayı unut" dönemi başladı.
2012	async / await	C# 5. Anders Hejlsberg PDC'de tanıttı. Callback hell'in sonu.
2016	ValueTask<T>	Hot path'lerde Task allocation'ı elemek için. Stephen Toub öncülük etti.
2018	IAsyncEnumerable	C# 8. "await foreach" — streaming işler nihayet async.
2020	async2 deneyi	Stephen Toub & Sergey Tepliakov: "ya state machine'i runtime'a taşısak?"
2026	Runtime Async	6 yıllık deney, ürüne dönüştü. .NET 11'de geliyor.

Peki neden 13 yıl sürdü?

01 .NET Framework'in mirası



CoreCLR ayrılana kadar runtime; **Windows, GAC, in-process** iki farklı GC, mixed-mode COM ile sarılıydı. Framework 4.8'de böyle bir ameliyat teknik olarak mümkün değildi — *ABI kırardı*.

02 CoreCLR'in olgunlaşması



.NET Core 1.0 (2016) → **.NET 5** (2020): önce cross-platform, sonra tiered JIT, sonra dynamic PGO. Runtime Async'in bindiği tüm altyapı ancak bu noktada hazır hale geldi.

Peki neden 13 yıl sürdü?

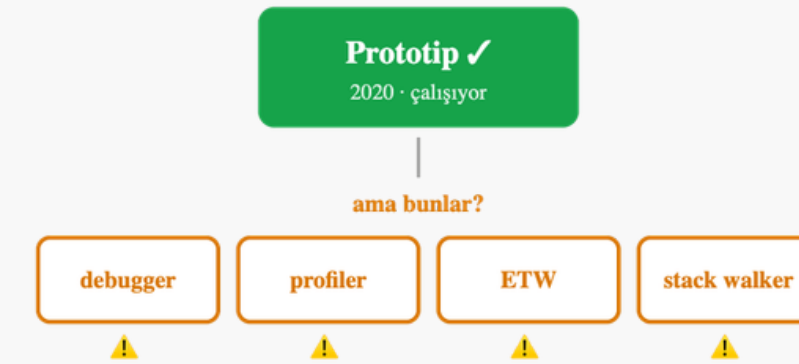
Migration sessiz olmak zorundaydı; prototip çalışıyordu ama *etrafındaki her şey* değildi.

03 Geriye dönük uyum



20 yıllık kod tabanı var. **Task**, **ValueTask**, **ConfigureAwait**, **SynchronizationContext** — hepsi çalışmaya devam etmek zorunda. "Her şeyi baştan yap" bir seçenek değildi.

04 Async2 deneyi



Stephen Toub & Sergey Tepliakov 2020'de prototipi yaptı. Sonuç: "*çalışıyor, ama debugger/profiler/ETW/stack walker hepsi yeniden öğrenmeli.*" 6 yıl oraya gitti.

Peki neden 13 yıl sürdü?

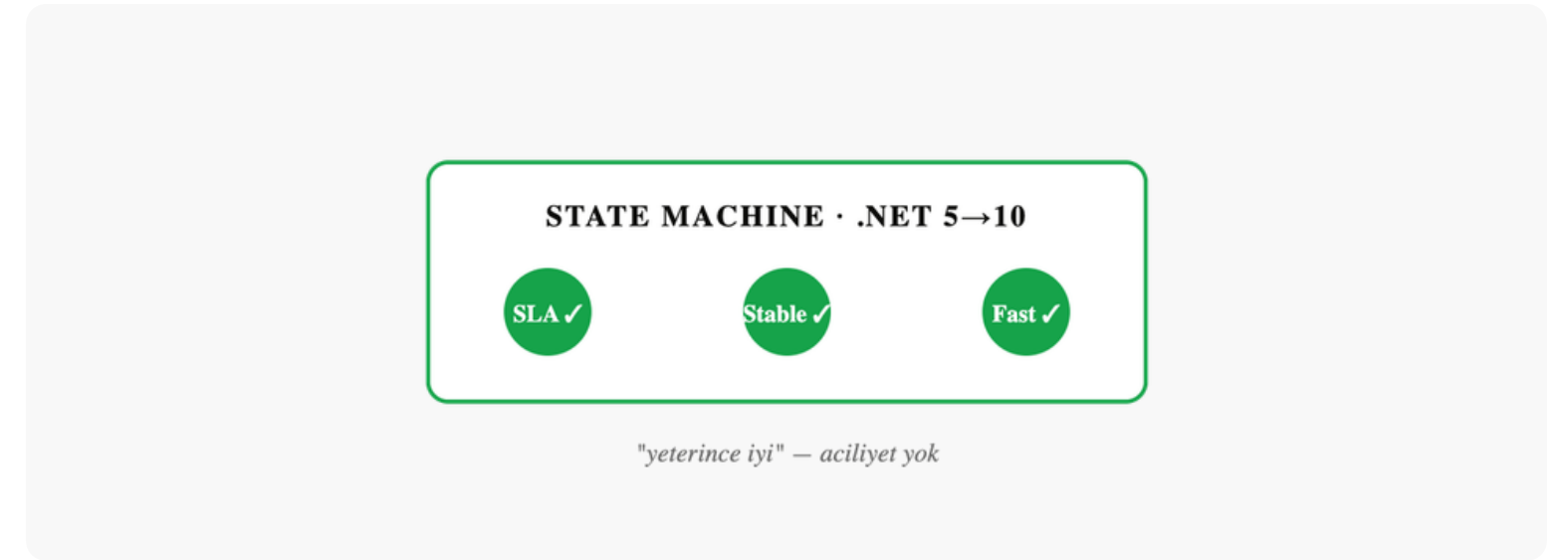
Son iki sebep teknik değil — stratejik.

05 Rakiplerin dersi



Go goroutine (2009), **Kotlin** coroutine (2018), **Java** Loom (2023) — herkes farklı yollardan geçti, farklı bedeller ödedi. .NET ekibi izledi, öğrendi, kendi yolunu seçti.

06 Kimse acele etmiyordu



Mevcut state machine modeli *"yeterince iyi"* çalışıyordu. Servisler ayakta, SLA tutuyor. Böyle bir değişiklik ancak runtime ekibi **"artık hazırız"** dediği gün başlar.

// neden 13 yıl • gerçek hayat metaforu

Çalışan binanın **temelini** değiştiremezsin.

// PROGRAM ÇALIŞIRKEN

	Hastane randevu sistemi • 7/24 açık	● AÇIK
	Banka şubesi gişe ekranı • 09:00'da herkes açar	● AÇIK
	Fabrika üretim hattı kontrol panosu	● AÇIK
	Market zinciri kasa + stok • binlerce şube	● AÇIK
	Muhasebe ofisi Excel + add-in • vergi dönemi	● AÇIK

.NET Framework • binanın temeli

.NET Framework, binlerce kuruma **aynı anda hizmet veren** bir bina gibi.

Hastane randevu alıyor. Banka 09:00'da açıyor. Fabrika hat çalıştırıyor. Market kasayı tutuyor. Muhasebeci vergi beyannamesi yazıyor.

Microsoft 2010'da gelip "**temeli yeniliyoruz**" diyemezdi.

// çözüm

Çözüm: **yan parsele yeni bina.**

Eskiye yıkmak yerine, yan boş parsele **yeni bir bina** diktiler: **CoreCLR**. Eski bina olduğu yerde durdu.

// ESKİ BINA - OLDUĞU YERDE



.NET Framework 4.8 - donduruldu

YAVAŞ TAŞINMA
10+ yıl boyunca, istiyorsanız

// YENİ BINA - BOŞ İNŞA EDİLDİ



CoreCLR - modern temel - her şey yeniden yazılabilir

Yeni binada kimse yoktu. **Async'i ancak orada** sıfırdan yazabilirlerdi — temeli sökmek için odaların boş olması gerekiyordu.

// SONUÇ

Yani ”~~geç kaldılar~~” değil—
doğru zamanı
beklediler. |

13 yılın sonunda **GC, JIT, PGO ve debug altyapısı** aynı anda hazır oldu. Bir özellik için *altı tane başka özelliğin* olgunlaşması gerekiyormuş.

// 101 • bir örnekle

Async = bekleme anlarını kullanmak.

Kafede kahve ısmarladığını düşün.



Senkron kod

Kahveni ısmarlarsın, barista yapana kadar tezgâhta bekler, sonra masaya geçersin.



Async kod

Kahveni ısmarlarsın, masaya geçer başka işe bakarsın. Hazır olunca barista seni çağırır.

Kod da aynı. `await` = "ben beklerken, sen başka işe bak."

Senin 5 satırın, arkada **bir sınıf** oluyor.

```
SEN YAZIYORSUN • 5 SATIR
async Task<int> Fetch()
{
    var a = await Step1();
    var b = await Step2(a);
    return a + b;
}
```

DERLENİYOR



DERLEYİCİ ÜRETİYOR • 1 SINIF

```
class FetchStateMachine
• int state // 0, 1, 2... • int a, b //
local'ler, artık field • Awaiter awaiter //
bekleme objesi • TaskBuilder builder

+ MoveNext() // devam noktası
```

Her await bir **devam noktası** . Local değişkenlerin sınıfın field'ına taşınıyor – çünkü metod geri dönerse onları stack'te tutamaz.

// bellek • 1. durum

Senkron dünya: iş **stack**'te başlar, stack'te biter.

STACK

çağrı sırasında üst üste büyür, dönünce küçülür.

Step1()

ben burayı geziyorum

Fetch()

Step1'i bekliyor

Main()

Fetch'i bekliyor

— stack tabanı —

HEAP

senkron kod için: boş.

(async nesne yok)

| Step1 bitince stack'ten düşer, kontrol Fetch'e geri gelir. Basit, lineer, ucuz.

// bellek • 2. durum • await anı

Async'te ilk **await** → iş **heap**'e çıkar.

STACK

await'ten sonra: bom boş.

(Fetch geri döndü – thread serbest, başka işe bakıyor)

HEAP

state machine burada asılı bekliyor.

FetchStateMachine

state = 0

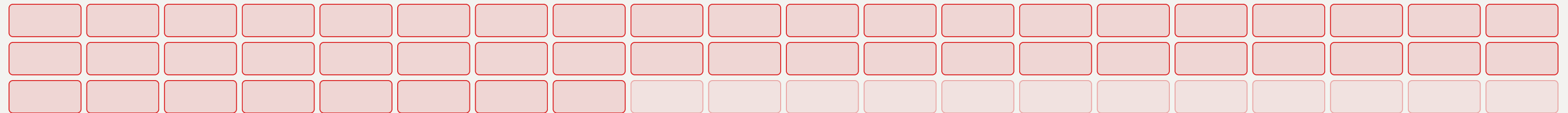
• a = 42 // local'den field'a taşındı • awaiter → Step2'nin Task'i • kim hazır olunca tetikleyecek?

| 1 await = 1 heap nesnesi. Sade görünüyor ama arkada her seferinde yeni bir nesne doğuyor.

// bellek • problem

1,000 istek = 1,000 kutu heap'te.

HEAP • SANIYEDE 10.000 İSTEK GELEN BİR ASP.NET CORE SERVİSİ



... ve devam ediyor

~80 B

her state machine — kendi başına küçük

800 KB/sn

10K RPS bir endpoint için sürekli alloc

Gen0 GC

sürekli toplanıyor → CPU + pause

Aynı kod. Aynı await. Heap **bom boş.**

STACK

runtime stack'i askıya alıp geri getiriyor.

Fetch()

askıda – runtime tutuyor

↓ Step2 hazır olunca buradan devam

HEAP

artık derleyici sınıfı üretmiyor.

✓ sıfır alloc

Çünkü **state machine** üretimi CoreCLR'ın kendi işi oldu. Sen aynı kodu yazıyorsun – runtime altta farklı anlıyor.

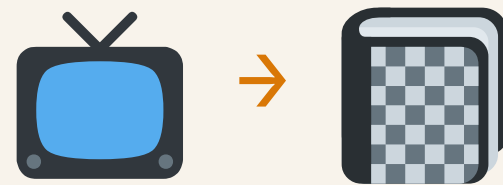
// peki, nasıl?

Runtime, kodu duraklatıyor.

Film izlerken  **pause** 'a bastığında ne yapıyorsun? Hiçbir şey. Film olduğu yerde kalıyor,  play'e basınca oradan devam ediyor. Async de böyle olmalıydı — nihayet oldu.

ESKİ YOL • .NET 10

Filmi durdurmak için **her kareyi deftere çizdirmek** .



Derleyici senin yerine `class`'a çeviriyordu .
Her `await` = yeni heap objesi.

YENİ YOL • .NET 11

 **bas.** Film olduğu yerde.  **bas.**
Devam.



CoreCLR işi kendi üstleniyor . Sen sadece  diyorsun (`await`).

II'nin adı: **stack frame suspension**.

Runtime, thread'in stack çerçevesini olduğu gibi askıya alıp geri getirebiliyor.



- 1 await'e gelindiğinde thread'in stack frame'i olduğu yerde durdurulur.
- 2 I/O biter, runtime bir sinyal alır.
- 3 Aynı frame yeniden uyandırılır — local'ler hâlâ yerli yerinde.
- 4 Hiç heap objesi üretilmez (genelde).

Aslında yeni bir fikir değil.

Stack frame suspension büyük dillerin hepsinde var, sadece farklı isimlerle.



| // Geç kaldı ama **doğru zamanda geldi** — diğer runtime'ların ayarlarını görerek. Bu avantaj.

// aynı bug • iki runtime • terminal çıktısı

Eski trace vs yeni trace.

```
stacktrace • .NET 10

$ dotnet run
Unhandled exception. System.InvalidOperationException:
  SQLite Error 5: database is locked
    at Data.Repo.SaveAsync(Order o)
    at AsyncTaskMethodBuilder`1.AwaitUnsafeOnCompleted(...)
    at Task.ExecuteWithThreadLocal(ref Task, Thread)
    at Repo.<SaveAsync>d__7.MoveNext()
    at Services.Checkout.RunAsync(CartId id)
    at TaskAwaiter.HandleNonSuccess(...)
    at Checkout.<RunAsync>d__3.MoveNext()
    at Api.OrdersController.Post(OrderDto o)

# 9 frame görüldü • 5 tanesi benim kodum değil
```

Bug nerede? **5 frame state-machine kalabalığı** tarıyorsun.

```
stacktrace • .NET 11

$ dotnet run
Unhandled exception. System.InvalidOperationException:
  SQLite Error 5: database is locked
    at Data.Repo.SaveAsync(Order o)
    at Services.Checkout.RunAsync(CartId id)
    at Api.OrdersController.Post(OrderDto o)

# 4 frame • hepsi senin kodun • bug: Repo.SaveAsync
```

Bug burada. **Repo.SaveAsync** — tek bakışta.

// aynı kod • sadece SDK sürümü değişti • dotnet/runtime Runtime Async (Preview)

// debugger'da nasıl görünüyor?

Stack trace, artık **gerçek hikâyeyi** anlatıyor.

.NET 10 • 13 FRAME

```
at InnerAsync>g__0_2()
at AsyncTaskMethodBuilder.Start
at Program.InnerAsync()
at MiddleAsync>d__1.MoveNext()
at ExceptionDispatchInfo.Throw
at TaskAwaiter.ThrowForNonSuccess
at TaskAwaiter.HandleNonSuccess
at TaskAwaiter.GetResult()
at MiddleAsync>d__1.MoveNext()
at AsyncTaskMethodBuilder.Start
at Program.MiddleAsync()
at OuterAsync>d__0.MoveNext()
at Program.Main()
```

.NET 11 • 4 FRAME

```
at Program.InnerAsync()
    in Program.cs:line 24

at Program.MiddleAsync()
    in Program.cs:line 18

at Program.OuterAsync()
    in Program.cs:line 12

at Program.Main()
```

| // Breakpoint'ler artık runtime-async metodların *içine* bağlanıyor. await boundary'lerinde step-over kolay.

\$ dotnet run -c Release --filter CheckoutAsync

BenchmarkDotNet v0.13 · .NET 10 (runtime-async opt-in) vs .NET 11

METRIC	RUNTIME	VALUE	Δ
Süre	.NET 10		842 ns
	.NET 11		318 ns -62%
Bellek	.NET 10		184 B
	.NET 11		0 B -100%
Stack	.NET 10		9 frame
	.NET 11		4 frame -56%

\$ █

// eski ⇌ yeni

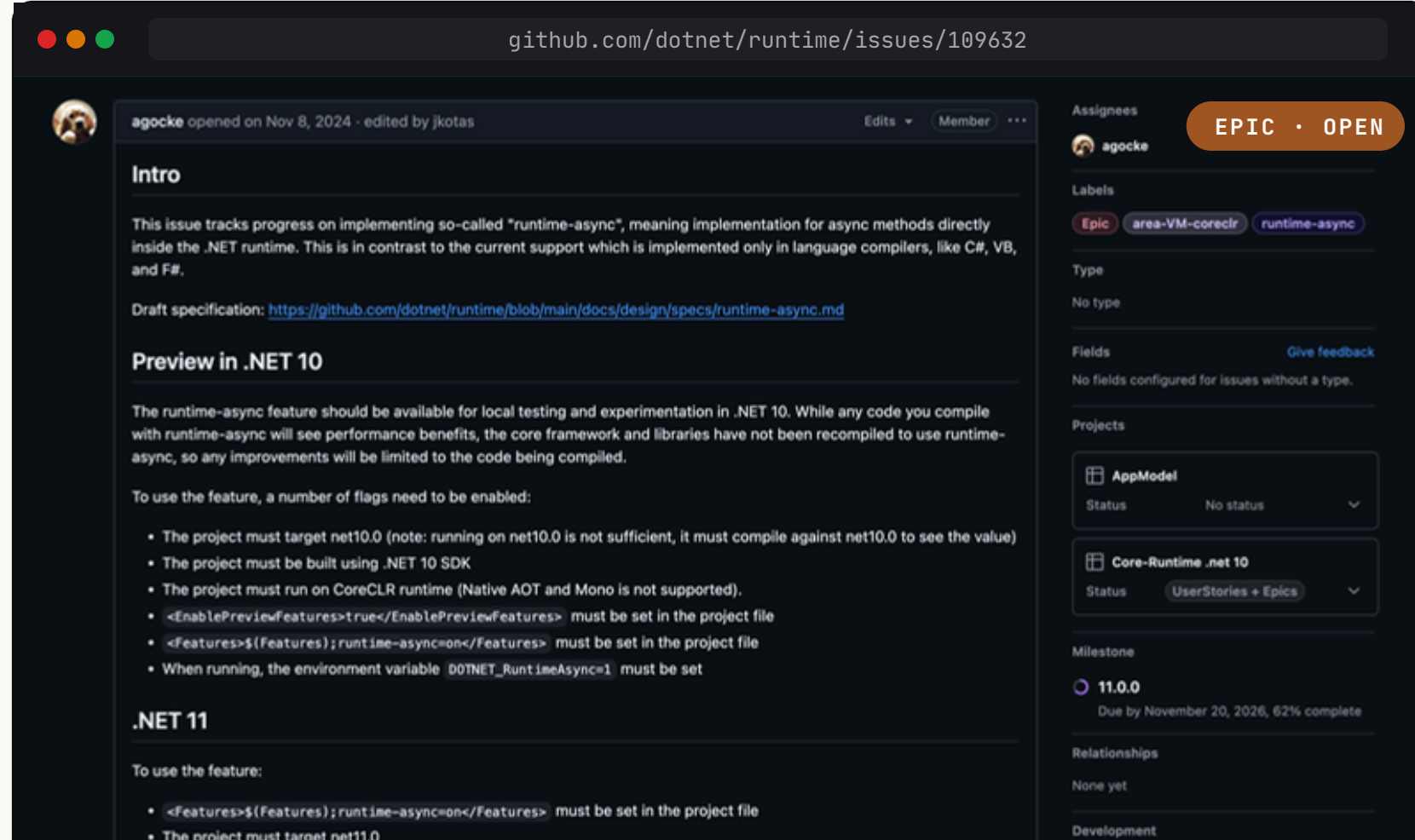
Sen değişmiyorsun. Çalışma şekli değişiyor.

KONU	.NET 10 VE ÖNCESİ	.NET 11 • RUNTIME ASYNC
State machine nerede üretilir?	Derleyici — C# seviyesinde	Runtime — CoreCLR seviyesinde
await için nesne gerekiyor mu?	Evet — MoveNext'li bir sınıf	Hayır — runtime stack'i askıya alır
Senin kodun değişiyor mu?	—	Hayır, aynı async/await
Stack trace nasıl görünür?	13+ frame — MoveNext ağırlıklı	4 frame — kendi metodların
Hot path'te alloc?	Var	Yok (genelde)
Debugger deneyimi?	Adım atlatma zor	Senkron gibi adım atlama

// Kaynak: dotnet/runtime #109632 "Runtime Async" epic (agocke, Nov 2024) • docs/design/specs/runtime-async.md

// kanıt • github.com/dotnet/runtime

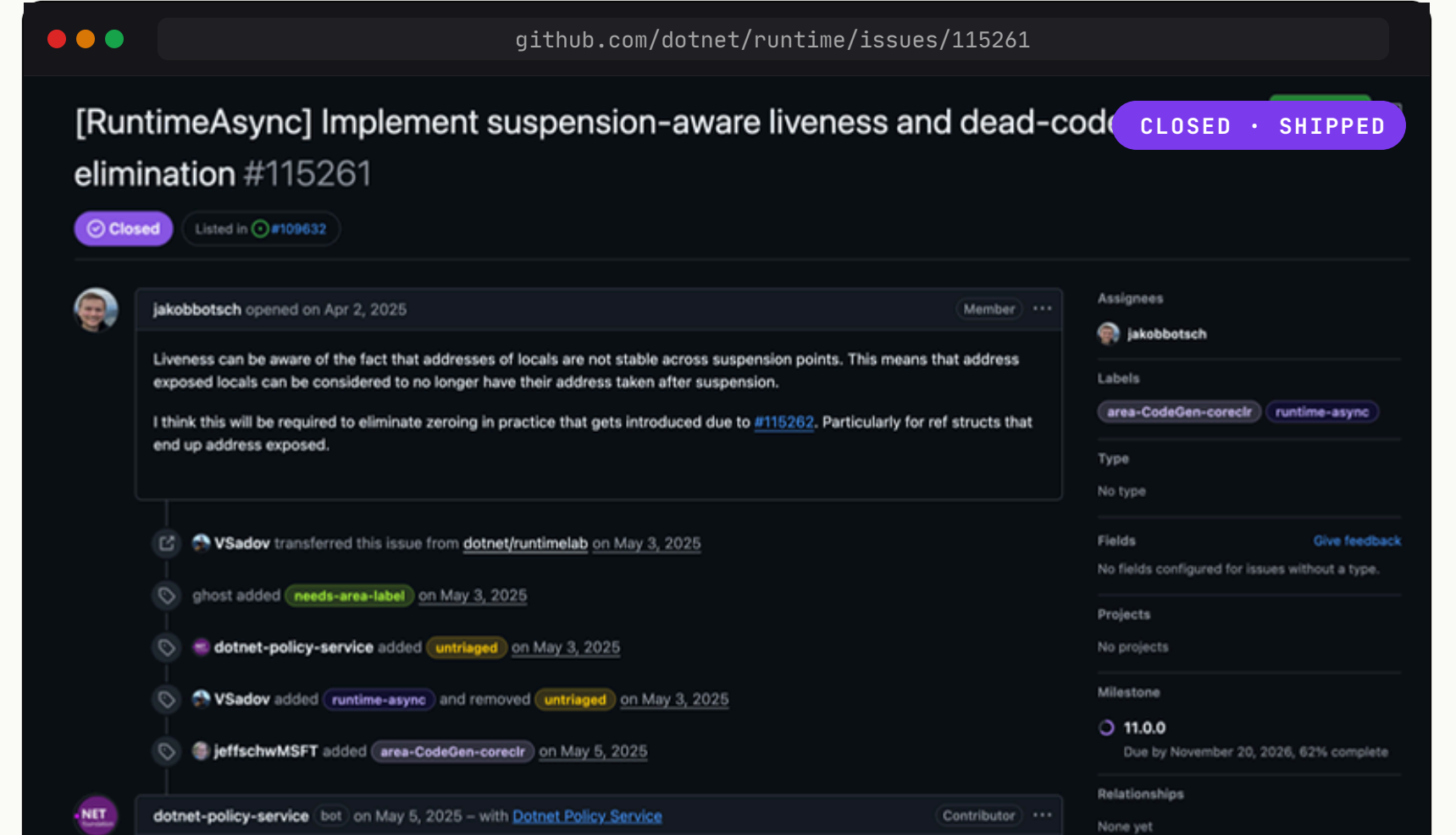
Planlanmıyor. Zaten yazıldı.



Runtime Async tracking epic

agocke • Nov 8, 2024 • Milestone .NET 11

Plan ve spec burada — resmi, public, sürüyor.



Suspension-aware liveness

jakobbotsch • Apr 2025 → Closed • Listed in #109632

Epic'teki alt-issue'lardan biri. Kod merge edildi.

40+

alt-issue epic'e bağlı — R2R, Native AOT, PGO, byref local zeroing, suspension-aware liveness...

Bu bir *demo* değil. Microsoft ekibinin .NET 11 için taahhüt ettiği feature.

// panik yok • breaking change değil

Kodun **aynen** çalışıyor.

DEĞİŞMİYOR

- ✓ async / await syntax'ı — aynı
- ✓ Task / ValueTask API'ları — aynı
- ✓ Var olan kodun recompile gerektirmez
- ✓ ConfigureAwait(false) hâlâ geçerli
- ✓ Library'lerin NuGet paketleri çalışır

GÖZE BATABILIR

- Stack trace parse eden kodun varsa kontrol et
- MoveNext() arayan logger/aspect'ler
- Source generator'ların async IL beklentileri
- Harmony / bazı profilerlar davranış farkı görebilir
- SynchronizationContext davranışı (takip altında)

| // Değişen şey: ürettiği IL ve runtime davranışı. Senin yazdığın şey değil.

Ne yapacaksın? İşte liste.

Bugün

5 dk



.NET 11 Preview'ı dene

CI'a ekstra bir SDK, mevcut branch'ı bozmaz.



Analyzer aç

Sync-over-async: `.Result`, `.Wait()`.

Bu hafta

2-3 saat



Stack trace'leri test et

Logger, Datadog, AppInsights, test asserter'ları.

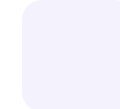


ValueTask hot path'lerini kıyasla

BenchmarkDotNet + MemoryDiagnoser. Gen0 düşüyor mu?

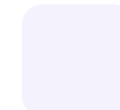
Bu sprint

1-2 gün



CancellationToken audit

Her async metoda — projede sıfır tolerans kuralı.



OTel → built-in'e geç

Kod azaldı, davranış aynı. Paketi çıkar, bekleme.

// tek cümlede

**.NET 11, async'i 10 yıl sonraki ilk büyük
tazelemesiyle yeniden doğurdu.**

Syntax'ın aynı. Alışkanlığın aynı. **Arkadaki makine başka.**

// ilk deneme

Test Edelim ...



\$./talk --end



TÜM HESAPLARIM

muratdinc.dev

github · x · linkedin · blog

Teşekkürler. Sorular? ■

“Aynı kodu yaz. Runtime senin için daha iyisini anlasın.”

– bu konuşmanın özeti

Murat Dinç

Microsoft MVP · Developer Technologies

github · /muratdinc x · @muratdinc